Natural Language Processing (Fall-2026)

Assignment 2, Part 2

Embedding Models, TF-IDF & Advanced Word Representations

Fall 2026 | FAST-NU Lahore

Instructor: Dr. Hajra Waheed

Teacher Assistant

Sidra Saleem: l258009@lhr.nu.edu.pk

For assignment & Viva-related queries, please contact the TA directly via email.

Submission Guidelines:

- Submit your assignment on Google Classroom in the format `"20XX.ipynb"` zip files.
- The deadline is **October 25, 2026, at 11:59 PM**. No extensions will be granted.

Declarations:

- Late submissions will incur penalties: 50% deduction on the first day and zero marks thereafter.
- Plagiarism will result in zero marks for the assignment.
- This is an individual assignment; collaboration or group work is strictly prohibited.
- Please ensure that you submit your own original work.

VIVA Policy:

- A VIVA (oral examination) will be conducted to assess your understanding of the assignment.
- The VIVA will be scheduled separately, and you will be notified of the date and time.
- Failure to attend the VIVA will result in zero marks for the assignment.

Academic Integrity:

- Plagiarism, collusion, and academic dishonesty will not be tolerated.
- Any instances of academic misconduct will be reported to the authorities and may result in severe penalties.

Total Marks: 75

Marks Breakdown:

- **Part 1 — Embedding Models** (30 marks) — All students
- **Part 2 — TF-IDF for Document Retrieval & Analysis** (20 marks) — All students
- **Part 3 — Advanced Word Representations** (25 marks) — PhD & MS 3rd Semester students ONLY

Instructions:

- You may use NumPy, NLTK, Gensim, matplotlib, and scikit-learn where specified. Any other library must be justified.
- All TF-IDF and PPMI computations must be implemented from scratch (do NOT use sklearn's TfidfVectorizer or similar).
- Include all requested tables, figures, and discussion answers as comments or printed output in your code.
- Each task specifies its deliverables — missing deliverables will result in mark deductions.

Part 1: Embedding Models [30 Marks]

This part covers distributional word representations: from sparse count-based vectors (TF-IDF, PPMI) to dense learned embeddings (Word2Vec) and Neural Language Models. You will implement, train, evaluate, and compare these representations using both intrinsic (word similarity) and extrinsic (text classification) evaluation paradigms.

Dataset: Use NLTK's Reuters corpus — categories: `'earn', 'acq', 'money-fx', 'grain'`. Combine all documents from these four categories.

Task P1T1 — Corpus Preparation and TF-IDF Representation [4 Marks]

Prepare a text corpus for embedding experiments and build TF-IDF vectors from scratch.

Steps:

- **1.** Load and preprocess the Reuters corpus: lowercase, remove punctuation, remove stopwords, tokenize.
- **2.** Filter vocabulary: keep only words with frequency ≥ 3 across the corpus.
- **3.** Construct a TF-IDF matrix **from scratch** (do NOT use sklearn's TfidfVectorizer — implement the TF-IDF formula manually).
- **4.** Report: vocabulary size, corpus size (total documents and tokens), and sparsity percentage of your TF-IDF matrix.

Deliverables:

- Vocabulary size and corpus statistics.
- TF-IDF matrix dimensions and sparsity percentage.
- Show TF-IDF vectors for 3 sample documents (display top-10 highest TF-IDF terms per document).

Task P1T2 — Co-occurrence Matrix and PPMI [5 Marks]

Build a word-word co-occurrence matrix from the preprocessed corpus using a symmetric context window.

Steps:

- **1.** Build a co-occurrence matrix using context window sizes **k = 3** and **k = 7**.
- **2.** Transform the raw co-occurrence counts into **Positive Pointwise Mutual Information (PPMI)** values.
- **3.** For 5 query words (choose words relevant to the Reuters financial domain, e.g., `market, trade, bank, price, stock`), find the top-5 nearest neighbors under both window sizes using cosine similarity on PPMI vectors.

Deliverables:

- Co-occurrence matrix dimensions for each window size.
- PPMI transformation: explain the formula and show a small example (3×3 submatrix before and after PPMI).
- Nearest-neighbor tables for 5 query words at k=3 and k=7.
- **Discussion (3–5 sentences):** How does changing the window size affect the type of similarity captured (syntactic vs. semantic)?

Task P1T3 — Word2Vec with Skip-Gram and Negative Sampling [5 Marks]

Train Word2Vec models using the Skip-Gram with Negative Sampling (SGNS) architecture on your preprocessed Reuters corpus.

Steps:

- **1.** Train two SGNS models with embedding dimensions: **d = 75** and **d = 150**.
- **2.** Use 8 negative samples and a context window of 5.
- **3.** You may use `gensim.models.Word2Vec` for training.
- **4.** For the same 5 query words from P1T2, report the top-5 nearest neighbors from each model.

Deliverables:

- Training configuration summary (dimensions, window, negative samples, epochs).
- Nearest-neighbor tables for 5 query words for both d=75 and d=150 models.
- **Discussion (3–5 sentences):** How do the neighbors from Word2Vec differ from those produced by PPMI? Which captures more semantic similarity?

Task P1T4 — Neural Language Model [6 Marks]

Build a simple feedforward Neural Language Model (NLM) that learns to predict the next word given a fixed-size context window, following the Bengio et al. (2003) architecture.

Steps:

- **1.** From your preprocessed Reuters corpus, create training samples: for each word in the corpus, use the previous **n = 4** words as input and the current word as the target.
- **2.** Build a feedforward neural language model in PyTorch or TensorFlow with:
  - An embedding layer (dimension d = 100, shared across all context positions)
  - Concatenation of the n context embeddings into a single vector (400-dimensional)
  - One hidden layer with 256 units and `tanh` activation
  - An output layer with softmax over the vocabulary
- **3.** Train the model for at least 5 epochs using **cross-entropy loss**. Use a learning rate of 0.001 with Adam optimizer. Track and plot the training loss curve.
- **4.** Compute **perplexity** on a held-out 20% test split: `PPL = exp(average cross-entropy loss)`
- **5.** Extract the learned embedding matrix from the model. For the same 5 query words from P1T2, report the top-5 nearest neighbors using cosine similarity on NLM embeddings.
- **6.** Test next-word prediction: given these 4-word contexts, report the model's top-5 predicted next words with their probabilities:
  - `"the stock market is"`
  - `"interest rates will be"`
  - `"the company reported a"`

Deliverables:

- Model architecture summary (layers, dimensions, parameters).
- Training loss curve over epochs.
- Test set perplexity.
- Nearest-neighbor tables for the 5 query words using NLM embeddings.
- Next-word predictions for the 3 test contexts.
- **Discussion (3–5 sentences):** How do the NLM embeddings compare to your Word2Vec embeddings from P1T3? Does the language modeling objective capture different word relationships than the Skip-Gram objective? What are the limitations of this fixed-window approach?

Task P1T5 — Intrinsic Evaluation (Word Similarity) [5 Marks]

Evaluate your embedding models using an intrinsic word similarity task.

Steps:

- **1.** Use the **SimLex-999** dataset (or WordSim-353 if SimLex is unavailable).
- **2.** Filter the dataset to include only word pairs where both words exist in your vocabulary.
- **3.** Compute cosine similarity between each word pair using:
    - (a) PPMI vectors (k=3)
    - (b) PPMI vectors (k=7)
    - (c) Word2Vec d=75
    - (d) Word2Vec d=150
    - (e) NLM embeddings (from P1T4)
- **4.** Compute **Spearman rank correlation** between your model's similarity scores and the human-annotated similarity scores.
- **5.** Report correlation values for all 5 models. Report the number of word pairs evaluated (after filtering).

Deliverables:

- Number of word pairs evaluated (after vocabulary filtering).
- Spearman correlation for each of the 5 models.
- Show 10 example word pairs with human scores and model scores side by side.
- **Discussion (3–5 sentences):** Which model correlates best with human judgments? Why?

Task P1T6 — Extrinsic Evaluation (Text Classification) [5 Marks]

Evaluate your embeddings on a downstream text classification task using the Reuters corpus categories as labels.

Steps:

- **1.** Construct document-level embeddings using two approaches:
    - **(a) Mean Pooling:** average all word embeddings in a document.
    - **(b) SIF (Smooth Inverse Frequency):** weighted average using word probabilities, with first principal component removal.
- **2.** Split data into 80% train / 20% test.
- **3.** Train a Logistic Regression classifier (from sklearn) on document embeddings.
- **4.** Report accuracy, precision, recall, and F1-score (macro-averaged) for each embedding type: PPMI (k=3), Word2Vec (d=75), Word2Vec (d=150).

Deliverables:

- Classification results table (accuracy, precision, recall, F1 for each embedding + pooling combination).
- Confusion matrix for the best-performing model.
- **Discussion (3–5 sentences):** Which embedding + pooling combination works best? Why does SIF outperform (or underperform) mean pooling?

Part 2: TF-IDF for Document Retrieval & Analysis [20 Marks]

In this part, you will build a complete TF-IDF-based information retrieval system from scratch and analyze its properties in depth. While you computed TF-IDF matrices in Part 1 as input for embeddings, here you will implement a full retrieval pipeline and explore document similarity and clustering.

Dataset: Use NLTK's Reuters corpus — all documents from the categories: `'earn', 'acq', 'money-fx', 'grain', 'crude', 'trade'`. This gives you a diverse multi-topic corpus for retrieval experiments.

Task P2T1 — TF-IDF Implementation from Scratch [7 Marks]

Implement the complete TF-IDF computation from scratch without using any library's built-in TF-IDF functions.

Steps:

- **1.** Preprocess all documents: lowercase, remove punctuation, remove stopwords, tokenize.
- **2.** Compute Term Frequency (TF) using **log-normalized TF**:

  `TF(t, d) = 1 + log (count(t, d)) if count > 0, else 0`

- **3.** Compute Inverse Document Frequency (IDF):

  `IDF(t) = log (N / df(t))`

where N = total number of documents, df(t) = number of documents containing term t.

- **4.** Compute: `TF-IDF(t, d) = TF(t, d) × IDF(t)`
- **5.** Filter vocabulary: keep only terms with `df ≥ 2` and `df ≤ 90% of N` (remove terms that are too rare or too common).

Deliverables:

- Final vocabulary size after filtering.
- TF-IDF matrix dimensions.
- For 5 sample documents, show the top-10 terms by TF-IDF score — do these terms accurately describe the document's topic?
- Top-10 highest IDF terms and top-10 lowest IDF terms in the corpus — **interpret** what high and low IDF means.

Task P2T2 — Document Retrieval System [7 Marks]

Build a document retrieval system using your TF-IDF representation and cosine similarity.

Steps:

- **1.** Implement a query function: given a free-text query, convert it to a TF-IDF vector using the same vocabulary and IDF values.
- **2.** Compute cosine similarity between the query vector and all document vectors.

- **3.** Return the top-10 most relevant documents for each query.
- **4.** Test with the following 5 queries:
    - `"oil prices and crude exports"`
    - `"stock market earnings report"`
    - `"foreign exchange rates dollar"`
    - `"wheat grain agricultural trade"`
    - `"company merger acquisition deal"`

Deliverables:

- For each query: list top-10 retrieved documents with their similarity scores and category labels.
- Compute **retrieval precision**: what fraction of the top-10 results belong to the expected category?
- **Discussion (3–5 sentences):** Which queries produced the best retrieval results? Which performed poorly and why?

Task P2T3 — Document Similarity and Clustering [6 Marks]

Use your TF-IDF vectors to explore inter-document similarity and perform basic document clustering.

Steps:

- **1.** Compute a pairwise cosine similarity matrix for a random sample of 50 documents (ensure at least 5 documents from each category).
- **2.** Visualize the similarity matrix as a **heatmap** (sorted by category).
- **3.** Apply **K-Means clustering** (K = 6, matching the number of categories) to the TF-IDF vectors of the sampled documents.
- **4.** Compare cluster assignments to actual category labels. Compute **purity**: for each cluster, what fraction of its documents belong to the most common category in that cluster?

Deliverables:

- Similarity heatmap (saved as PNG).
- Cluster assignments vs. actual categories (printed table).
- Purity score for the overall clustering.
- **Discussion (3–5 sentences):** Do documents from the same category cluster together? Which categories are hardest to separate and why? What are the limitations of TF-IDF for clustering compared to dense embeddings?

Part 3: Advanced Word Representations [25 Marks]

THIS PART IS FOR PhD AND MS 3rd SEMESTER STUDENTS ONLY

This part explores advanced embedding methods that go beyond Word2Vec. You will work with GloVe (global co-occurrence statistics), FastText (subword information), and Contextual Embeddings. Each method addresses a specific limitation of the models you implemented in Part 1.

Dataset: Same Reuters corpus as Parts 1 and 2.

Task P3T1 — GloVe: Global Co-occurrence Embeddings [10 Marks]

GloVe learns embeddings by factorizing the log of the global word-word co-occurrence matrix. Unlike Word2Vec (which learns from local context windows one at a time), GloVe captures corpus-wide co-occurrence statistics directly.

Steps:

- **1.** Using the co-occurrence matrix you built in P1T2 (window k=7), implement the **GloVe objective function** from scratch:

$$J = \Sigma \quad f(X \quad) \times (w \quad w \quad + b \quad + b \quad - \log X \quad)^2$$

where $f(x) = (x/x_max)^{\alpha}$ if $x < x_max$, else 1 is the weighting function (use $\alpha = 0.75$, x_max = 100).

- **2.** Train GloVe embeddings with dimension **d = 100** using SGD or AdaGrad. Train for at least 50 epochs. Track and plot the loss curve.
- **3.** For the same 5 query words from P1T2, report the top-5 nearest neighbors using cosine similarity on GloVe vectors.
- **4.** Compute the Spearman rank correlation on the SimLex-999 dataset (same methodology as P1T4) for your GloVe embeddings.

Deliverables:

- GloVe loss curve over training epochs.
- Nearest-neighbor tables for the 5 query words.
- Spearman correlation on SimLex-999.
- **Discussion (5–8 sentences):** Compare GloVe results against your PPMI and Word2Vec results from Part 1. Does GloVe capture different relationships? In what ways does the global co-occurrence objective produce different embeddings than the local-context Skip-Gram objective? Where does GloVe fail on this corpus (consider corpus size)?

Task P3T2 — FastText: Subword Embeddings [8 Marks]

FastText represents each word as a bag of character n-grams. This means it can generate embeddings for out-of-vocabulary (OOV) words — a major limitation of Word2Vec and GloVe.

Steps:

- **1.** Train a FastText model on the Reuters corpus using `gensim.models.FastText` with:
  - `embedding dimension = 100, window = 5, min_count = 3`
  - `character n-gram range: min_n = 3, max_n = 6`
- **2.** For the same 5 query words, report top-5 nearest neighbors.
- **3.** Test OOV (out-of-vocabulary) word handling. Generate embeddings for these **misspelled / unseen** words and find their nearest neighbors in the vocabulary:
  - `"trdaing"  (misspelling of "trading")`
  - `"finacial" (misspelling of "financial")`
  - `"stockmarket" (compound word)`
  - `"euro-dollar" (hyphenated term)`
  - `"cryptocurrency" (unlikely in Reuters corpus)`
- **4.** Compare: try to get Word2Vec embeddings for the same 5 OOV words. What happens?
- **5.** Run the same SimLex-999 Spearman correlation evaluation as P1T4.

Deliverables:

- Nearest-neighbor tables for the 5 query words.
- OOV word embeddings: for each misspelled/unseen word, show its top-3 nearest in-vocabulary neighbors.
- Spearman correlation on SimLex-999.
- **Discussion (5–8 sentences):** How does FastText handle OOV words that Word2Vec cannot? In what scenarios would subword embeddings be critical (think about morphologically rich languages, social media text, medical/legal terminology)? Compare FastText's SimLex correlation against Word2Vec and GloVe — which performs best and why?

Task P3T3 — Comprehensive Embedding Comparison [7 Marks]

Bring together all six embedding methods (TF-IDF, PPMI, Word2Vec, NLM, GloVe, FastText) and perform a systematic comparison.

Steps:

- **1.** Create a **master comparison table** with these columns: Method | Type (sparse/dense) | Handles OOV? | SimLex Spearman ρ | Best Classification F1 | Training Time (approx.).
- **2.** Perform the same extrinsic text classification evaluation from P1T6 (Logistic Regression on Reuters categories) using GloVe and FastText document embeddings (mean pooling). Add these results to your comparison table.
- **3.** Dimensionality reduction visualization:
  - Select 30 words from 3 semantic categories (10 words each): financial terms (e.g., stock, market, bond, equity…), agricultural terms (e.g., wheat, grain, corn, crop…), and geopolitical terms (e.g., government, trade, tariff, sanction…).
  - Use t-SNE to reduce embeddings to 2D for each of the 6 methods.
  - Create 6 scatter plots (one per method), coloring points by semantic category.
  - Save all 6 plots as a single figure with subplots.
- **4.** Analogy evaluation: test each dense embedding model (Word2Vec, NLM, GloVe, FastText) on these analogy tasks using vector arithmetic `v(king) − v(man) + v(woman) ≈ v(queen)`:
  - `"dollar" − "usa" + "japan" = ?`
  - `"stock" − "company" + "farm" = ?`
  - `"export" − "sell" + "buy" = ?`
  - `"oil" − "crude" + "grain" = ?`
  - Report the top-3 results for each analogy from each model.

Deliverables:

- Master comparison table (all 6 methods across all metrics).
- Classification results for GloVe and FastText (added to the table from P1T6).
- t-SNE visualization figure (6 subplots).
- Analogy results table (4 models × 4 analogies).
- **Discussion (8–10 sentences):** Write a critical analysis covering: (1) When would you choose sparse representations (TF-IDF/PPMI) over dense ones? (2) What does the t-SNE visualization reveal about how each method organizes semantic space? (3) Which method handles the Reuters domain best and why? (4) What are the fundamental trade-offs between count-based (TF-IDF, PPMI, GloVe) and prediction-based (Word2Vec, FastText) approaches? (5) If you were building an NLP system for a domain with rapidly evolving vocabulary (e.g., social media, biomedical), which method would you choose and why?